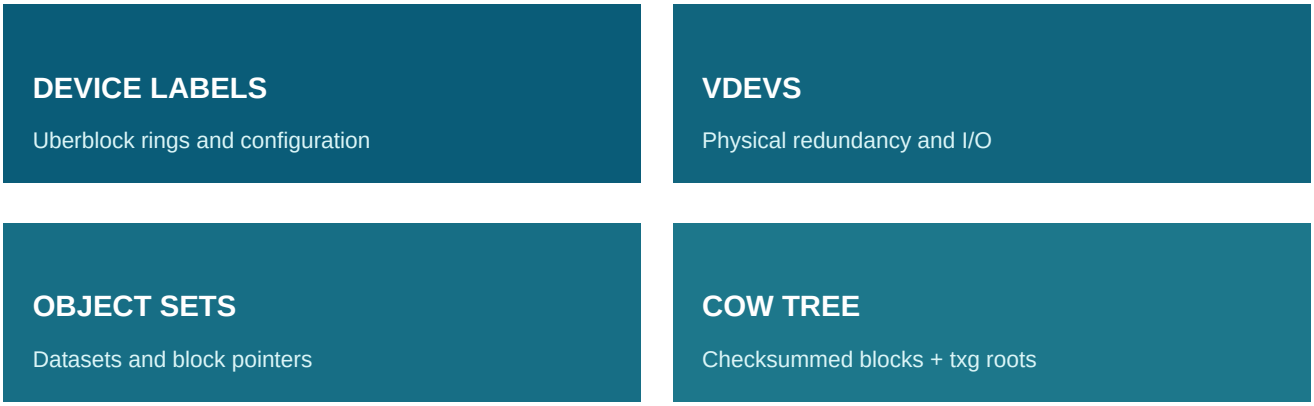
ZFS - pooled transactional filesystem

HISTORY / ORIGINAL DESIGN / DISK FORMAT

Sun’s pooled storage architecture combines checksummed copy-on-write trees, datasets, transaction groups and integrated redundancy policy.

This is a technical reference to the upstream filesystem’s historical design, not an installer manual. The format’s on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

ZFS - pooled transactional filesystem

HISTORICAL CHRONOLOGY

ZFS was developed at Sun Microsystems in the early 2000s and introduced in Solaris during 2005, challenging the traditional division between filesystems, volume managers and independent RAID layers. It was subsequently published under open-source licensing and evolved into OpenZFS across multiple operating systems. Its design emphasised end-to-end verification, large capacity and administrative simplification.

The modern OpenZFS ecosystem supports pools and datasets rather than a single simple filesystem volume. Snapshots, clones, compression, replication and multiple redundancy arrangements emerged from an integrated transactional model. Exact feature flags and pool format capabilities must be read from the installed implementation; historical Solaris and modern OpenZFS releases are not identical.

ZFS - pooled transactional filesystem

ORIGINAL ON-DISK ARCHITECTURE

ZFS pools contain one or more top-level virtual devices, each constructed from disks, mirrors, RAIDZ groups or other supported layouts. Device labels record configuration and uberblock rings. An uberblock describes a committed root of a tree of object sets, block pointers and data. Block pointers contain physical locations, sizes, birth transaction and checksums, so they verify a child from outside the child block itself.

Copy-on-write means modified blocks are allocated anew and then reachable through newly published parent pointers. Transaction groups write the new tree and commit through a valid uberblock. Dataset objects, metadata and indirect blocks form the logical namespace. Physical block layout and redundancy cannot be inferred from filename paths or a single-device offset.

ZFS - pooled transactional filesystem

ALLOCATION AND DIRECTORIES

ZFS manages allocation at the storage pool level using metaslabs and space-map structures. Allocators select free regions from the pool and map writes through vdev geometry. Files are described by DMU objects and block-pointer trees. Datasets provide separate properties, quotas and mount points; zvols expose block devices rather than ordinary directory trees.

Snapshots reference past consistent trees; clones add a writable descendant, often sharing physical blocks. Compression and deduplication change physical space consumption. Copy-on-write metadata may increase write amplification for small random writes, but makes block sharing and replication practical.

ZFS - pooled transactional filesystem

INTEGRITY AND RECOVERY

ZFS verifies data and metadata using block-pointer checksums. When redundant good copies exist, a read or scrub can replace corrupted blocks. A checksum can detect corruption without being able to repair it if no valid replica survives. Synchronous writes use the ZFS Intent Log (ZIL), while normal asynchronous changes are grouped in txg commits. The ZIL does not describe a universal redo log of all file data.

On import, ZFS chooses a valid committed uberblock and reconstructs the last consistent pool view. Pools have labels and redundant copies of critical metadata, but pool import, damaged-vdev recovery and resilvering are version- and topology-dependent. There is no traditional fsck equivalent that rewrites all metadata for a classic non-COW image.

ZFS - pooled transactional filesystem

DESIGN RATIONALE AND LIMITATIONS

ZFS integrates previously separate responsibilities to address silent data corruption, fragmentation across large arrays and complexity of volume management. The costs include memory use for caching and metadata, COW write amplification, free-space pressure, and dependence on vdev layout decisions that are not always reversible.

A single ZFS.so shared library does not thereby become a complete pooled-storage engine. Pool import, device discovery, crash-consistent writing, resilvering, scrub, ARC management, ZIL and vdev fault handling require much more than a parser that recognises an uberblock.

ZFS - pooled transactional filesystem

INFILTRATOROS INTEGRATION BOUNDARY

The historical ZFS format and OpenZFS implementation remain independent of the InfiltratorOS filesystem catalogue entry. If our ZFS.so exposes only experimental or read-only operations, the PDF must not imply complete production Linux support. The optional ‘.ko‘ cannot be claimed qualified without actual kernel module evidence.

Defragmentation is especially different from classic FAT: the ZFS allocator owns future physical placement. A generic promise to reserve an adjacent extent conflicts with transaction, dataset and vdev placement behaviour.

ZFS - pooled transactional filesystem

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. OpenZFS official copy-on-write and transaction-group architecture

https://openzfs.github.io/openzfs-docs/Basic%20Concepts/Copy-on-write.html

2. OpenZFS official basic concepts and vdev/dataset reference

https://openzfs.github.io/openzfs-docs/Basic%20Concepts/

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

ZFS / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read the pool’s virtual-device configuration from device labels. The physical disks, mirror or RAIDZ arrangement must be known before translating any logical block.

2. Inspect the uberblock rings and select a valid committed transaction-group state; an older valid root may remain available after an interrupted update.

3. Follow the committed uberblock to the Meta Object Set, then to datasets, object sets and their dnodes rather than expecting a traditional fixed inode table.

4. Decode each block pointer’s logical and physical sizes, birth transaction, checksum and compression information to locate and validate the child block.

5. Use metaslab allocation maps and spacemaps to interpret free space. A file’s pathname is not enough to determine whether a particular physical sector is in use.

6. Recognise that multiple datasets and snapshots can retain earlier block versions. Reclaiming a block requires resolving all live references.

7. Distinguish data vdevs from log, cache, spare and special allocation classes; not all devices in a pool store the same kind of persistent user data.

8. Interpret ZAP objects, indirect blocks and bonus buffers in the context of each object type. A generic B-tree parser cannot directly replace the ZFS object model.

ZFS / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Never choose a newer-looking uberblock without verifying its checksum, transaction group and reachable object graph.

2. Check block-pointer checksums on the correct logical or physical bytes as specified by the checksum and compression algorithms.

3. Distinguish pool-level vdev redundancy from dataset-level copies. One readable block is not evidence that all replicas or RAIDZ stripes are sound.

4. Import and feature compatibility require knowledge of supported features-for-read and active feature flags; a higher on-disk feature set may prohibit writes.

5. An interrupted transaction may leave blocks allocated but unreachable from the committed uberblock; they are not automatically live filesystem contents.

6. Verify the ashift and sector alignment constraints of the vdev configuration. Logical record size and underlying physical sector size are different concepts.

7. Snapshots and clones can retain old block pointers; do not classify those blocks as lost merely because the current head dataset no longer references them.

8. Keep pool import, scrub, resilver and destructive repair separate. Each operates on different evidence and carries different risk.